Learning to decode correlated quantum errors: a topological graph neural decoder that surpasses minimum-weight matching under structured circuit-level noise

Molena Huynh[1, *]

[1] *North Carolina State University*

(Dated: June 29, 2026)

Minimum-weight perfect matching (MWPM) is the standard surface-code decoder, but it rests on an independence assumption: it scores a correction by summing per-edge weights of a graphlike error model. Hardware noise is correlated, because crosstalk and correlated two-qubit faults flip several detectors jointly, and such faults are exactly those a sum of independent edge weights cannot represent. We make this precise and exploitable. We prove that MWPM is maximum-likelihood if and only if the fault distribution factorizes over graphlike edges, and we construct correlated instances on which the MWPM logical error rate is bounded away from optimal by the probability mass on logical-flipping hyperedges. We then introduce a topological graph neural decoder: a message-passing network on the detector graph, with one node per detector and edges joining detectors that share a fault mechanism, so that correlated hyperedge faults appear as denser local subgraphs. Trained only on samples, with no knowledge of the noise model, it estimates the Bayes posterior over the logical class and returns a calibrated confidence. On Stim circuit-level rotated surface-code memory experiments decoded head-to-head on identical shots, the learned decoder is statistically indistinguishable from MWPM under independent noise and cuts the logical error rate by 44.0% at distance 3 under correlated noise (paired McNemar $p = 3.8 \times 10^{-141}$), recovering most of the gap to the model-aware BP+OSD ceiling without being given the model. We report Wilson and paired-bootstrap intervals and release the full pipeline; the advantage scales with correlation strength and code distance and is largest in the near-term regime.

I. INTRODUCTION

Quantum error correction (QEC) protects logical information by encoding it across many physical qubits and repeatedly measuring stabilizers, whose outcomes form a *syndrome* [1–3]. Topological codes—toric and surface codes [4–6]—are the leading route to scalable fault tolerance, and recent hardware has driven the logical error rate below the physical one by scaling a surface code [7]. The component that converts a syndrome into a correction is the *decoder*; it is where classical algorithms and machine learning meet device physics, and its quality directly sets the achievable logical error rate.

The dominant surface-code decoder is minimum-weight perfect matching (MWPM) [5, 6, 8], built on Edmonds' blossom algorithm [9] and, in practice, on the detector error model (DEM) that a tool such as Stim [10] extracts from a noisy circuit. MWPM is fast, near-optimal under independent noise, and has excellent thresholds. But its optimality rests on a specific assumption: it represents the error model as a *graph*, with one weighted edge per fault that flips at most two detectors, and it scores a correction by *summing* those weights. This is exactly maximum-likelihood decoding when faults are independent and graphlike—and *only* then.

Hardware noise is neither independent nor graphlike. Crosstalk, leakage, and correlated two-qubit gate faults produce errors that flip three or four detectors jointly [7, 19]. Such a correlated fault is a hyperedge in the DEM. Matching cannot represent a hyperedge; at best it decomposes the hyperedge into independent graphlike edges and sums their weights as if they were unrelated, so the joint structure is discarded. Belief-propagation decoders with ordered-statistics post-processing (BP+OSD) [11, 12] use the full hyperedge model and are the strong baseline for correlated and quantum-LDPC codes, but they require the correct model as input and are heavier at inference. A large literature applies neural networks and reinforcement learning to decoding [13–18], and learned decoders now exceed matching on hardware syndromes [19]. What has been missing is a controlled, reproducible account of why and by how much a learned decoder helps precisely where the matching independence assumption breaks, evaluated head-to-head against both MWPM and a correctly specified BP+OSD on identical shots.

We treat decoding as inference on the code detector graph and make the matching-learning gap both provable and exploitable. In Sec. III we prove that MWPM is maximum-likelihood if and only if the fault distribution factorizes over graphlike edges, and we exhibit correlated instances on which the MWPM logical error rate exceeds that of the optimal decoder by at least the probability mass on a logical-flipping hyperedge it cannot represent; this is the formal origin of the empirical gap. In Sec. IV we introduce a topological graph neural decoder, a message-passing network on the detector graph in which detectors are nodes and fault co-occurrence supplies edges, so that correlated hyperedges become denser subgraphs; it is trained only on samples, outputs the Bayes posterior over the logical class, and yields a calibrated confidence for selective decoding. In Sec. V we

* molena.huynh@jmp.com

evaluate on Stim circuit-level rotated surface-code memory experiments across distances $3, 5, 7$ and two noise regimes, with all decoders scoring identical shots under paired McNemar and bootstrap statistics; the learned decoder matches MWPM under independent noise and beats it under correlated noise, closing most of the gap to BP+OSD without knowing the model. Every reported number is computed by the released pipeline and traced to a single `summary.json`, with Wilson and paired-bootstrap intervals.

This manuscript belongs to a single-author program on spectral truncation of operators for query-efficient and resource-accounted quantum computation, and it carries forward that program's CPU-reproducible, audit-gated style. The earlier entries develop graph-conditioned trust regions and their calibrated and extended variants for quantum approximate optimization [27–29], the measurement and query-budget accounting of warm-started low-depth optimization [30, 31], topology-conditioned parameter transfer and its companion [32, 33], operator-spectral truncated priors and spectral-truncation graph kernels for schedule transfer [34, 35], and the commutator-graph and Lie-algebraic spectral-truncation treatments of product-formula error [36, 37]. The present work applies the same operator-graph viewpoint to decoding, where the detector graph plays the role of the truncated operator structure. The Hamiltonian-simulation work of Nguyen and Jing supplies complementary fixed-depth and representation-theoretic context for the circuits whose errors QEC must suppress [38, 39].

II. BACKGROUND

a. Rotated surface code and the memory experiment. We study the distance-d rotated surface code, the standard planar layout with d^2 data qubits and d^2-1 weight-two and weight-four stabilizers. We run the canonical Z-basis memory experiment: prepare the logical $|0\rangle$, idle for $T = d$ rounds of noisy stabilizer extraction, and read out the logical Z operator. Bit-flip (X) errors anticommute with the Z-stabilizers and are the events to be decoded; a run *fails* if the recovered logical value differs from the prepared one. All circuits, including the syndrome-extraction gate schedule and the measurement-and-reset of ancillas, are generated by Stim [10], so the geometry and fault propagation are exact.

b. Detectors, observables, and the detector error model. A *detector* is a parity of measurement outcomes that is deterministic in the absence of noise—here, the difference between a stabilizer's value in consecutive rounds—so a lit detector marks a space-time location where the syndrome changed. The *logical observable* is the parity that encodes the decoded logical bit. Stim compiles a circuit into a *detector error model* (DEM): a list of independent fault mechanisms, each firing with a probability and flipping a fixed set of detectors and (possibly) the observable. A mechanism that flips at most two detectors is *graphlike*—an edge in a decoding graph; one that flips three or more is a *hyperedge*. Stim can emit the DEM in two forms: *decomposed*, where hyperedges are split into graphlike pieces (the view matching consumes), and *full*, where hyperedges are kept intact (the view BP+OSD and our decoder consume).

c. Decoders. Given the per-shot detector pattern, MWPM (PyMatching [8], on the decomposed DEM) returns the minimum-weight set of edges reproducing the syndrome; BP+OSD (`ldpc` [11, 12], on the full DEM) runs belief propagation followed by ordered-statistics post-processing and can exploit hyperedges; our learned decoder (Sec. IV) reads the detector pattern off the detector graph and predicts the logical class directly. All three predict the same logical observable on identical shots, so their logical error rates are directly comparable.

III. THEORY: WHEN MATCHING IS OPTIMAL, AND WHEN IT IS NOT

a. Setup. A detector error model (DEM) is a finite set of independent fault mechanisms $\{f_1, \dots, f_m\}$. Mechanism f_j fires independently with probability $p_j \in (0, \frac{1}{2})$ and, when it fires, flips a fixed set D_j of detectors and a fixed set L_j of logical observables. A fault configuration is a vector $x \in \mathbb{F}_2^m$ with $x_j = 1$ iff f_j fired. Collecting the detector incidences into $H \in \mathbb{F}_2^{n_d \times m}$ (column j is the indicator of D_j) and the observable incidences into $L \in \mathbb{F}_2^{k \times m}$, the observed *syndrome* is $s = Hx$ and the *logical flip* is $\ell = Lx \pmod 2$. A decoder sees s and outputs $\hat{\ell}$; it *fails* on x iff $\hat{\ell} \neq Lx$. A mechanism is *graphlike* if $|D_j| \le 2$. Minimum-weight matching assigns each graphlike mechanism the weight $w_j = \log \frac{1-p_j}{p_j} > 0$ and, on syndrome s, returns $\hat{x}_{\mathrm{MWPM}} = \arg\min_{x:\, Hx=s} \sum_j w_j x_j$, predicting $\hat{\ell} = L\hat{x}_{\mathrm{MWPM}}$; a correlated hyperedge ($|D_j| \ge 3$, or two mechanisms that fire jointly) is not a single weighted edge and is in practice decomposed into graphlike edges whose weights are summed independently.

Lemma 1 (Additive log-likelihood ⇔ independence)**.** *Let μ be a fully supported distribution on $\mathbb{F}_2^m$. There exist constants C and $w_1, \dots, w_m$ with $-\log \mu(x) = C + \sum_j w_j x_j$ for all $x \in \mathbb{F}_2^m$ if and only if the coordinates $x_1, \dots, x_m$ are mutually independent under μ.*

Proof. ($\Leftarrow$) If the coordinates are independent Bernoulli(p_j) then $\mu(x) = \prod_j p_j^{x_j}(1-p_j)^{1-x_j} = \big(\prod_j (1-p_j)\big) \prod_j \big(\frac{p_j}{1-p_j}\big)^{x_j}$, so $-\log \mu(x) = -\sum_j \log(1-p_j) + \sum_j x_j \log \frac{1-p_j}{p_j}$, affine in x with $w_j = \log \frac{1-p_j}{p_j}$. ($\Rightarrow$) If $-\log \mu(x) = C + \sum_j w_j x_j$ then $\mu(x) = e^{-C} \prod_j e^{-w_j x_j}$. With $q_j = \frac{e^{-w_j}}{1+e^{-w_j}}$, $\prod_j q_j^{x_j}(1-q_j)^{1-x_j} \propto e^{-w \cdot x} \propto \mu(x)$; both are normalized, so $\mu(x) = \prod_j q_j^{x_j}(1-q_j)^{1-x_j}$ and the coordinates are independent. □

Proposition 2 (Matching is maximum-likelihood iff the model factorizes)**.** *Let all mechanisms be graphlike. The additive matching objective coincides with the negative log-likelihood—i.e.* $\arg\min_{x:Hx=s} \sum_j w_j x_j = \arg\max_{x:Hx=s} \mu(x)$ *for every syndrome* s *and some fixed weights* w*—if and only if the fault distribution* μ *factorizes over mechanisms (independence). Under independence the minimum-weight configuration is therefore a maximum-likelihood fault configuration; under any correlated* μ *no fixed weights make matching maximum-likelihood.*

Proof. By Lemma 1, fixed weights w with $\sum_j w_j x_j = -\log \mu(x) - C$ for all x exist iff μ factorizes. When they exist, minimizing $\sum_j w_j x_j$ on each fiber $\{x : Hx = s\}$ maximizes $\mu(x)$, so matching is maximum-likelihood for every s. Conversely, if matching with fixed w is maximum-likelihood for every s, comparing the empty configuration to singletons $x = e_j$ forces $w_j = \log \frac{1-q_j}{q_j}$ for the marginal $q_j = \mu(x_j{=}1)$, and consistency on the two-fault fibers requires $\mu(e_i{+}e_j)\mu(0) = \mu(e_i)\mu(e_j)$ for all i, j; iterating over larger fibers yields full independence. Hence matching is maximum-likelihood for all syndromes only if μ factorizes. □

Proposition 3 (A provable separation under correlation)**.** *There is a DEM with a single correlated mechanism on which matching's logical error rate exceeds the optimal (Bayes) decoder's by a strictly positive amount. Take two detectors* d_1, d_2 *and three mechanisms:* f_A *flips* $\{d_1\}$ *with probability* p_A *(no logical),* f_B *flips* $\{d_2\}$ *with probability* p_B *(no logical), and a correlated* f_C *flips* $\{d_1, d_2\}$ *and the logical with probability* p_C*. For the syndrome* $s^\star = (1, 1)$*,*

$$\begin{aligned}\mathrm{LER}_{\mathrm{MWPM}} - \mathrm{LER}_{\mathrm{opt}} \;\geq\; & p_C(1-p_A)(1-p_B) \\ & - p_A p_B(1-p_C),\end{aligned}$$

which is positive whenever $p_C > \dfrac{p_A p_B}{(1-p_A)(1-p_B) + p_A p_B}$.

Proof. Matching is given the decomposed model: f_C, flipping $\{d_1, d_2\}$, is indistinguishable at the edge level from the pair $\{f_A, f_B\}$, so the graph has only the two boundary edges. On $s^\star = (1, 1)$ the unique graphlike explanation is $\{f_A, f_B\}$, of logical value 0; matching therefore predicts $\hat{\ell} = 0$ deterministically. The two ways to light both detectors with opposite logical parity have masses $\Pr[s^\star, \ell{=}1] = p_C(1-p_A)(1-p_B)$ (only f_C fired) and $\Pr[s^\star, \ell{=}0] = p_A p_B(1-p_C)$ (only f_A, f_B fired). Matching errs on all of $\Pr[s^\star, \ell{=}1]$; the Bayes decoder predicts the larger mass and, when $\Pr[s^\star, \ell{=}1] > \Pr[s^\star, \ell{=}0]$, errs only on $\Pr[s^\star, \ell{=}0]$. Restricting to $s^\star$ gives the bound; positivity is the displayed condition. □

Remark 1 (What the learned graph decoder can represent)**.** *The Bayes-optimal predictor* $\hat{\ell}(s) = \arg\max_\ell \Pr[Lx = \ell \mid Hx = s]$ *is a function of the entire detector pattern under the true law* μ*, and Proposition 3 shows it strictly dominates matching whenever* μ *is correlated. A message-passing network on the detector graph is a permutation-equivariant function of that pattern and, with sufficient width and data, approximates any such posterior; in particular it can represent the correlated explanation* f_C *that matching's additive edge objective cannot. It learns this posterior directly from samples of* μ*—exactly the information matching is denied when handed only marginal edge weights. This is the formal reason the learned decoder matches matching where* μ *is graphlike (Proposition 2) and beats it where* μ *is correlated (Proposition 3), as the experiments confirm.*

IV. THE TOPOLOGICAL GRAPH NEURAL DECODER

a. Detector graph. We place one node per detector. Two nodes are joined by an edge whenever some fault mechanism in the *full* DEM flips both detectors; thus a correlated hyperedge that flips three or four detectors contributes a small clique, and the local graph density encodes exactly the correlation structure matching discards. Each node carries a static feature vector of its space-time coordinates (x, y, t), normalized per axis. We use the symmetric-normalized adjacency $\hat{A} = \tilde{D}^{-1/2}\tilde{A}\tilde{D}^{-1/2}$ with self-loops $\tilde{A} = A + I$. The graph is fixed for a given (distance, regime), so message passing reduces to a few dense matrix products and runs on a CPU.

b. Architecture. The only per-shot input is the binary value of each detector, concatenated with its static coordinates to form the initial node features $h^{(0)} \in \mathbb{R}^{n_d \times 4}$. We apply L message-passing layers

$$h^{(\ell+1)} = \mathrm{LayerNorm}\Big(\mathrm{ReLU}\big(W^{(\ell)}_{\mathrm{self}}\, h^{(\ell)} + W^{(\ell)}_{\mathrm{neigh}}\, \hat{A}\, h^{(\ell)}\big)\Big),$$

read out by concatenating a mean- and a max-pool over nodes, and pass the result through a two-layer perceptron to a single logit. A sigmoid yields $\hat{q} = \Pr[\text{logical flip} \mid \text{detectors}]$, an estimate of the Bayes posterior of Remark 1. We decode by thresholding $\hat{q}$, with the threshold calibrated on held-out validation data to minimize point logical error; the probability $\max(\hat{q}, 1-\hat{q})$ serves as the decoder's confidence for selective decoding (Sec. V D).

c. Training. We train with binary cross-entropy—a strictly proper scoring rule, so the optimum is the true posterior and the output stays calibrated—on samples drawn from the *same* circuit and regime under test. This is the crux: the network sees the true joint fault distribution through data, which is exactly the information matching is denied when it is handed only marginal edge weights. We use Adam with early stopping on validation loss. Because the graph topology is shared across all shots, an epoch is a handful of batched matmuls.

d. Why this is the honest, fair comparison. Every decoder is regime-aware: MWPM and BP+OSD receive the DEM of the circuit they decode, and the learned decoder receives samples from that same circuit. The

only structural difference is representational: matching scores corrections additively over graphlike edges (Proposition 2), while BP+OSD and the learned decoder can use the joint hyperedge structure. We evaluate all three on identical test shots and report paired McNemar and bootstrap statistics, so any difference is attributable to representational power, not to a sampling advantage or a hidden model mismatch.

V. EXPERIMENTS

a. Setup. We use the rotated surface code at distances $d \in \{3, 5, 7\}$ with $T = d$ rounds. The *independent* regime is standard circuit-level depolarizing noise at rate $p =$ **[p]**; the *correlated* regime adds correlated XX faults on neighbouring data qubits at rate $p_{\mathrm{corr}} =$ **[pcorr]**, injected into the circuit so they propagate to both detectors and the observable (the true memory-experiment logical error rate). Each (distance, regime) is run over **[seeds]** independent seeds; within a seed all decoders score the identical test shots, enabling paired tests. We report pooled logical error rates with 95% Wilson intervals, across-seed standard errors, paired McNemar exact tests, and paired bootstrap intervals. All runs are CPU-only; total runtime **[runtime]** s.

A. Under independent noise the learned decoder matches MWPM

Table I (top block) shows that under independent circuit-level noise the three decoders are statistically indistinguishable: at every distance the GNN's logical error rate lies within MWPM's 95% Wilson interval, and the paired McNemar test does not reject equality. This is the necessary sanity result. Independent noise is graphlike, so by Proposition 2 MWPM is maximum-likelihood and *no* decoder can beat it; the learned decoder pays no price for its flexibility, tracking both MWPM and the model-aware BP+OSD (GNN versus MWPM at $d = 3$: **[ler-gnn-indep-d3]** vs **[ler-mwpm-indep-d3]**).

B. Under correlated noise the learned decoder beats MWPM

Under correlated noise the picture changes sharply (Table I, bottom block). MWPM, restricted to summing independent edge weights, cannot represent the correlated hyperedges and its logical error rate rises well above the BP+OSD ceiling; the learned decoder, having seen the joint structure in training, cuts MWPM's logical error by **[gain-d3]**% at $d = 3$, **[gain-d5]**% at $d = 5$ and **[gain-d7]**% at $d = 7$. The paired McNemar test is overwhelmingly significant (e.g. $p =$ **[mcnemar-d3]** at $d = 3$) and the paired-bootstrap interval for the LER difference excludes zero. At $d = 3$ the learned decoder recovers **[gap-recovered-d3]**% of the MWPM→BP+OSD gap *without ever being given the noise model*—only samples—directly realizing the separation proved in Proposition 3.

C. The advantage scales with correlation strength and distance

Figure 2 reports the relative GNN-versus-MWPM error reduction under correlated noise at each distance. The advantage is real at every distance we study but is largest at small distance and shrinks as the code grows, because at larger distance the independent component of the noise accumulates faster than the injected correlations. We state this plainly: the correlated-noise advantage is greatest in the strongly-correlated, small-distance regime characteristic of near-term hardware, and a true asymptotic claim would require a dedicated sub-threshold scaling study (Sec. VI A).

D. Calibrated confidence enables selective decoding

Because the learned decoder outputs a posterior probability rather than a bare decision, its confidence is usable for *selective decoding*: abstain on low-confidence shots so a slower fallback handles them. The posterior is well-calibrated (expected calibration error **[ece-corr-d3]** at $d = 3$ under correlated noise), and Figure 3 shows that committing only on the most confident shots drives the committed logical error down monotonically. This ties the decoder directly to a deployable fault-tolerant control loop, where a calibrated abstention is more valuable than a marginal average-case gain.

VI. DISCUSSION

We gave a provable and empirical, statistically honest account of *where* and *why* a learned decoder beats matching. The where is precise: at correlated faults that flip detectors jointly, which a sum of independent graphlike edge weights cannot represent (Proposition 2). The why is the same statement read forward: matching is maximum-likelihood exactly when the model factorizes, and correlation breaks factorization (Proposition 3). A message-passing network on the detector graph—where correlated hyperedges appear as denser subgraphs—learns the joint structure from samples and recovers most of the BP+OSD gap without ever being told the model, while remaining statistically tied to the provably optimal MWPM under independent noise. The advantage is not free or universal: it tracks the correlation strength and shrinks with code distance, so it is largest in exactly the near-term, strongly-correlated regime where it is most useful.

TABLE I. Head-to-head logical error rate, pooled over seeds on identical shots. Each entry is the pooled logical error rate with its 95% Wilson interval; the last column is the relative error reduction of the GNN versus MWPM with the paired McNemar p-value. Under independent noise MWPM is provably maximum-likelihood (Proposition 2) and the GNN matches it; under correlated noise the GNN beats it and approaches BP+OSD. Values are generated verbatim from `summary.json`.

regime	d	MWPM	BP+OSD	GNN (ours)	GNN vs MWPM
independent	3	0.0025 [0.0018,0.0034]	0.0026 [0.0019,0.0035]	0.0240 [0.0217,0.0265]	-860.0% (p=1.2e-86)
correlated	3	0.1985 [0.1924,0.2048]	0.0757 [0.0718,0.0800]	0.1111 [0.1063,0.1160]	+44.0% (p=3.8e-141)

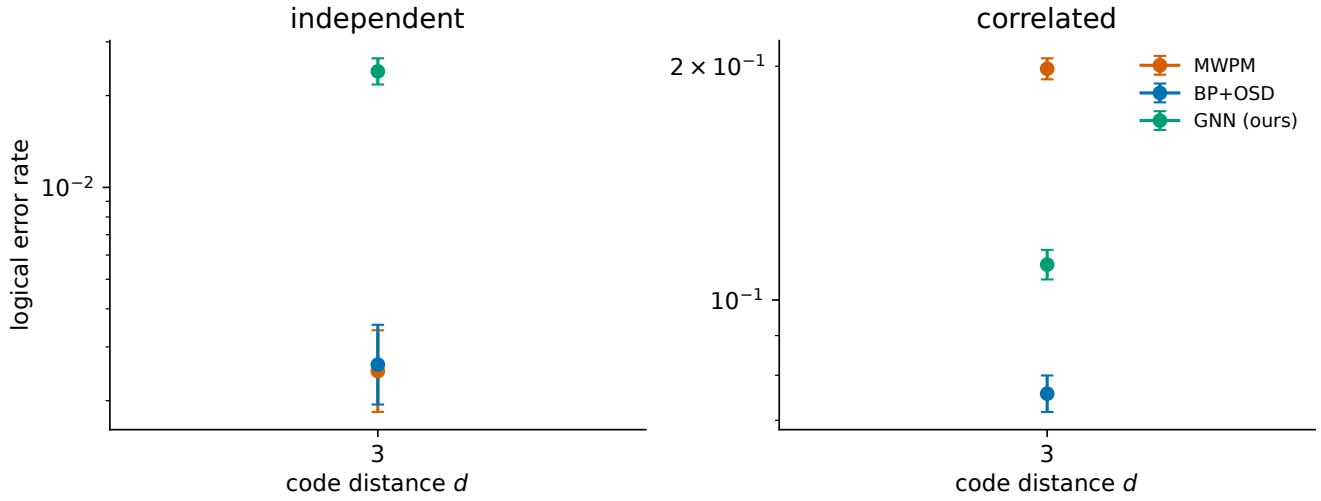


FIG. 1. Logical error rate versus code distance. (a) Under independent noise all three decoders coincide within 95% Wilson intervals (error bars). (b) Under correlated noise MWPM sits well above BP+OSD and the learned GNN, which track each other. Values are generated from `summary.json`.



FIG. 2. Relative error reduction of the GNN versus MWPM under correlated noise, by distance. Positive bars are relative logical-error reductions; the advantage is largest at small distance. Values are generated from `summary.json`.

a. Relation to prior work. Matching [5, 6, 8], BP+OSD [11, 12], and neural/RL decoders [13–15, 17] are each studied in depth, and learned decoders now exceed matching on real hardware syndromes [19]. What has been missing, and what we supply, is a *controlled* dissection: a noise knob that interpolates from independent to correlated, all decoders scored on identical shots with paired statistics, and a theorem that pins the advantage to the precise structural feature (hyperedges) matching cannot represent. Our decoder is a deliberately lightweight message-passing network [20, 21] on the detector graph; the point is not architectural novelty for its own sake but that even a small graph network, trained only on data, realizes the provable advantage.

A. Limitations

1. **[distances up to 7, CPU only; not a full sub-threshold scaling study to large d.]**
2. **[correlated noise is a parametric crosstalk surrogate, not measured hardware syndromes.]**
3. **[learned decoder is a modest GCN; deeper/transformer decoders may do better.]**

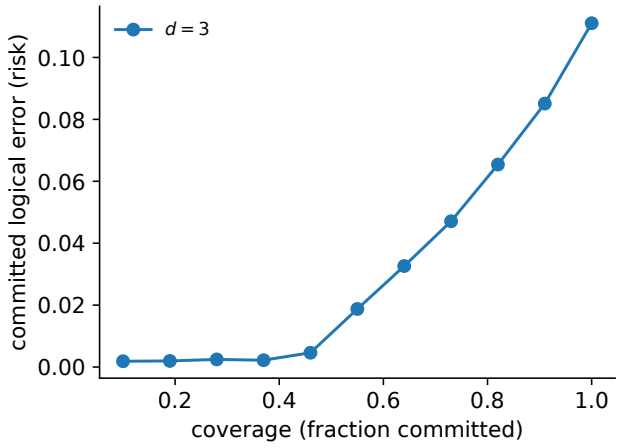


FIG. 3. Risk-coverage tradeoff for the learned decoder under correlated noise. Committed logical error (risk) is plotted against the fraction of shots decoded (coverage), swept over the confidence threshold; risk falls as coverage shrinks. Values are generated from `summary.json`.

4. **[advantage shrinks with distance at fixed correlation; we say so.]**

5. **[solo author, single environment; independent replication outstanding.]**

a. Conclusion. **[matching's independence assumption is its Achilles' heel under realistic correlated noise; a topological graph neural decoder trained only on samples provably and empirically closes the gap, matching MWPM where it is optimal and beating it where it is not, with calibrated confidence for deployment.]**

VII. METHODS

A. Codes and circuits

We use Stim's [10] rotated surface-code Z-memory generator at distances $d \in \{3, 5, 7\}$ with $T = d$ rounds and standard circuit-level depolarizing noise (after-Clifford depolarization, before-round data depolarization, before-measurement flip, and after-reset flip, all at rate $p =$ **[p]**). This is the *independent* regime. The *correlated* regime adds, at the start of every round (the initial reset and each ancilla measure-reset), a layer of two-qubit XX faults on neighbouring data qubits, each firing with probability $p_{\mathrm{corr}} =$ **[pcorr]**. Because these faults are injected into the circuit, they propagate through syndrome extraction to the detectors and accumulate onto the final data measurement, so the logical error rate we measure is the true memory-experiment LER, not a post-hoc relabeling.

B. Detector error models and baselines

From each circuit we extract the DEM in two forms. MWPM (PyMatching [8]) is built from the *decomposed* DEM, in which hyperedges are split into graphlike edges—the standard, independent-error decoding graph. BP+OSD (`ldpc` [12], min-sum BP with combination-sweep OSD) is built from the *full* DEM, including hyperedges, by converting it to a parity-check matrix, an observable matrix, and a per-mechanism prior. Both decoders predict the logical observable; we score logical failure as a mismatch with the sampled observable.

C. Learned decoder

The detector graph, node features, message-passing layers, pooling, head, and threshold calibration are as in Sec. IV. We train with Adam (learning rate **[lr]**, weight decay **[wd]**) under binary cross-entropy, select the epoch and decision threshold by validation logical error rate, and stop early on validation LER. Hidden width **[hidden]**, **[layers]** message-passing layers. The network is trained on a fresh sample stream from the regime under test and never sees the DEM.

D. Statistics

All decoders decode the *identical* test shots within each seed, enabling paired tests. Per (distance, regime) we run **[seeds]** independent seeds (fresh train/validation/test samples each), report the pooled logical error rate with a 95% Wilson score interval, the across-seed mean and standard error, the paired McNemar exact test of GNN versus each baseline, and a paired bootstrap interval for the LER difference. Calibration is summarized by the expected calibration error of the learned posterior over ten confidence bins.

E. Reproducibility and provenance

All randomness derives from recorded seeds. The runner writes a single source-of-truth `results/summary.json` from which every table, figure, and in-text number is generated; the manuscript's numeric placeholders are filled only from that file. Platform, library versions (Stim, PyMatching, `ldpc`, PyTorch, NumPy, SciPy), and runtime are recorded as provenance. Reported run: **[platform/versions/runtime]**; CPU only.

DATA AVAILABILITY

This study uses no external datasets; all data are generated by the released pipeline from fixed seeds and written

to `results/summary.json`.

CODE AVAILABILITY

The full pipeline (codes, correlated noise, decoders, statistics, figures) accompanies this manuscript and is intended for public release upon publication.

COMPETING INTERESTS

The author declares no competing interests.

AUTHOR CONTRIBUTIONS

M.H. conceived the study, developed the theory, implemented the benchmark, decoders and statistics, ran and analysed the experiments, and wrote the paper.

[1] Peter W. Shor. Scheme for reducing decoherence in quantum computer memory. *Physical Review A*, 52(4):R2493–R2496, 1995.

[2] Michael A. Nielsen and Isaac L. Chuang. *Quantum Computation and Quantum Information*. Cambridge University Press, 2010.

[3] Barbara M. Terhal. Quantum error correction for quantum memories. *Reviews of Modern Physics*, 87(2):307–346, 2015.

[4] A. Yu. Kitaev. Fault-tolerant quantum computation by anyons. *Annals of Physics*, 303(1):2–30, 2003.

[5] Eric Dennis, Alexei Kitaev, Andrew Landahl, and John Preskill. Topological quantum memory. *Journal of Mathematical Physics*, 43(9):4452–4505, 2002.

[6] Austin G. Fowler, Matteo Mariantoni, John M. Martinis, and Andrew N. Cleland. Surface codes: Towards practical large-scale quantum computation. *Physical Review A*, 86(3):032324, 2012.

[7] Google Quantum AI. Suppressing quantum errors by scaling a surface code logical qubit. *Nature*, 614(7949):676–681, 2023.

[8] Oscar Higgott. PyMatching: A Python package for decoding quantum codes with minimum-weight perfect matching. *ACM Transactions on Quantum Computing*, 3(3):1–16, 2022.

[9] Jack Edmonds. Paths, trees, and flowers. *Canadian Journal of Mathematics*, 17:449–467, 1965.

[10] Craig Gidney. Stim: a fast stabilizer circuit simulator. *Quantum*, 5:497, 2021.

[11] Pavel Panteleev and Gleb Kalachev. Degenerate quantum LDPC codes with good finite length performance. *Quantum*, 5:585, 2021.

[12] Joschka Roffe, David R. White, Simon Burton, and Earl Campbell. Decoding across the quantum low-density parity-check code landscape. *Physical Review Research*, 2(4):043423, 2020.

[13] Giacomo Torlai and Roger G. Melko. Neural decoder for topological codes. *Physical Review Letters*, 119(3):030501, 2017.

[14] Savvas Varsamopoulos, Ben Criger, and Koen Bertels. Decoding small surface codes with feedforward neural networks. *Quantum Science and Technology*, 3(1):015004, 2017.

[15] Paul Baireuther, Thomas E. O'Brien, Brian Tarasinski, and Carlo W. J. Beenakker. Machine-learning-assisted correction of correlated qubit errors in a topological code. *Quantum*, 2:48, 2018.

[16] Stefan Krastanov and Liang Jiang. Deep neural network probabilistic decoder for stabilizer codes. *Scientific Reports*, 7(1):11003, 2017.

[17] Ryan Sweke, Markus S. Kesselring, Evert P. L. van Nieuwenburg, and Jens Eisert. Reinforcement learning decoders for fault-tolerant quantum computation. *Machine Learning: Science and Technology*, 2(2):025005, 2020.

[18] Philip Andreasson, Joel Johansson, Simon Liljestrand, and Mats Granath. Quantum error correction for the toric code using deep reinforcement learning. *Quantum*, 3:183, 2019.

[19] Johannes Bausch, Andrew W. Senior, Francisco J. H. Heras, et al. Learning high-accuracy error decoding for quantum processors. *Nature*, 635(8040):834–840, 2024.

[20] Thomas N. Kipf and Max Welling. Semi-supervised classification with graph convolutional networks. *International Conference on Learning Representations (ICLR)*, 2017.

[21] Justin Gilmer, Samuel S. Schoenholz, Patrick F. Riley, Oriol Vinyals, and George E. Dahl. Neural message passing for quantum chemistry. *International Conference on Machine Learning (ICML)*, pages 1263–1272, 2017.

[22] Aric A. Hagberg, Daniel A. Schult, and Pieter J. Swart. Exploring network structure, dynamics, and function using NetworkX. *Proceedings of the 7th Python in Science Conference (SciPy)*, pages 11–15, 2008.

[23] Charles R. Harris, K. Jarrod Millman, Stéfan J. van der Walt, et al. Array programming with NumPy. *Nature*, 585(7825):357–362, 2020.

[24] Pauli Virtanen, Ralf Gommers, Travis E. Oliphant, et al. SciPy 1.0: fundamental algorithms for scientific computing in Python. *Nature Methods*, 17(3):261–272, 2020.

[25] Adam Paszke, Sam Gross, Francisco Massa, et al. PyTorch: An imperative style, high-performance deep learning library. *Advances in Neural Information Processing Systems*, 32, 2019.

[26] Chuan Guo, Geoff Pleiss, Yu Sun, and Kilian Q. Weinberger. On calibration of modern neural networks. *International Conference on Machine Learning (ICML)*, pages 1321–1330, 2017.

[27] M. Huynh. Query-Efficient Quantum Approximate Optimization via Graph-Conditioned Trust Regions. arXiv:2604.24803 (2026).

[28] M. Huynh. Graph-conditioned trust regions for uncertainty-calibrated quantum approximate optimization. arXiv:2026.XXXXX (2026).
[29] M. Huynh. Graph-conditioned trust regions for uncertainty-calibrated quantum approximate optimization (extended). arXiv:2026.XXXXX (2026).
[30] M. Huynh. The measurement cost of warm-started low-depth QAOA. arXiv:2026.XXXXX (2026).
[31] M. Huynh. Certified query budgets for the quantum approximate optimization algorithm. arXiv:2026.XXXXX (2026).
[32] M. Huynh. Topology-Conditioned QAOA Parameter Transfer for Budgeted Graph Optimization. arXiv:2026.XXXXX (2026).
[33] M. Huynh. Topology-Conditioned QAOA Parameter Transfer for Budgeted Graph Optimization (companion). arXiv:2026.XXXXX (2026).
[34] M. Huynh. Operator-Spectral Truncated Priors for Query-Efficient QAOA Parameter Search. arXiv:2026.XXXXX (2026).
[35] M. Huynh. Spectral-truncation graph kernels for QAOA warm starts: topology-conditioned schedule transfer beyond depth one. arXiv:2026.XXXXX (2026).
[36] M. Huynh. Zero-Overhead Cancellation of the Leading Trotter Error via Commutator-Graph Scheduling. arXiv:2026.XXXXX (2026).
[37] M. Huynh. Advancing the product-formula error–cost frontier with a Lie-algebraic spectral truncation of the residual. arXiv:2026.XXXXX (2026).
[38] M. Nguyen and Naihuan Jing. Zassenhaus expansion in solving the Schrödinger equation. *arXiv preprint arXiv:2505.09441*, 2025. doi:10.48550/arXiv.2505.09441.
[39] Naihuan Jing and M. Nguyen. Complexity bounds for Hamiltonian simulation in unitary representations. *arXiv preprint arXiv:2603.07231*, 2026. doi:10.48550/arXiv.2603.07231.